BreaQ 2026 — Challenge 1: Trust No Input

Application-Layer Metadata Tampering in Quantum Policy Verification

Team μ-Qbit (`DEMO-TEAM`)
Richard Watry & Dylane Mahazoasy

October 25, 2026

Executive Summary & Threat Model (0:00–0:45)

The Incident: Migrated and imported quantum assets (`MIG-DELTA`, `INCIDENT-7`) fail verification with `MISMATCH`. Operations blames "unreliable quantum circuits."

One-Sentence Security Claim

The failures are caused **not by quantum noise or circuit bugs**, but by an application-layer trust-boundary violation in `ChallengeService.execute()`, which passes untrusted manifest `compatibility_map` permutations into `compatibility_decode()` prior to authorization.

Trusted / Authoritative

- `registry_fingerprint` ($q_0 q_1 q_2 q_3$)
- `build_bv_probe` (BV oracle)
- Raw backend `counts` (512 shots)

Untrusted (Crosses Boundary)

- Manifest `compatibility_map`
- Adapter `reported_fingerprint`
- Breaks **Soundness** & **Completeness**

Phase A & B: Three-Boundary Fault Localization (0:45–1:45)

We inspected the 5-qubit Bernstein-Vazirani pipeline across **three independent boundaries**:

1. **Boundary 1 (Prepared Circuit & `Statevector`):** CX gates on $q_0 \ldots q_3 \rightarrow q_4$ prepare the exact registry oracle state with probability $p = 1.0$.
2. **Boundary 2 (Raw Backend `counts`):** 100% deterministic (512/512 shots). Reversing Qiskit's big-endian key ($c_3 c_2 c_1 c_0 \rightarrow q_0 q_1 q_2 q_3$) matches `registry_fingerprint` on **all 4 cases**.
3. **Boundary 3 (Application Report):** `compatibility_decode` permutes canonical bits using `compatibility_map`, corrupting valid outputs into `MISMATCH`.

Case ID	Raw Key (512/512)	Canonical ($q_0..q_3$)	Manifest Map	Reported → Decision
CAL-ALPHA	1001	1001 (Reg: 1001)	[0, 1, 2, 3]	1001 → **MATCH**
CAL-BETA	0110	0110 (Reg: 0110)	[0, 1, 2, 3]	0110 → **MATCH**
MIG-DELTA	0101	1010 (Reg: 1010)	[1, 0, 2, 3]	0110 → **MISMATCH**
INCIDENT-7	0011	1100 (Reg: 1100)	[3, 1, 2, 0]	0101 → **MISMATCH**

Phase B & C: Causal Proof & Minimal Reproducer (1:45–2:30)

Falsifiable Causal Claim: Holding the quantum circuit and registry constant while replacing `compatibility_map` with `[0, 1, 2, 3]` flips `MIG-DELTA` and `INCIDENT-7` from `MISMATCH` to `MATCH`.

Phase C — Minimal Reproducer (`source_fingerprint = '1000'`):

- Single active CX(q_0, q_4) gate (Hamming weight $w = 1$) under a single adjacent slot swap ($0 \leftrightarrow 1$):

Config	Source FP	`compatibility_map`	Raw Counts	Canonical	Reported / Decision
Identity	1000	`[0, 1, 2, 3]`	{0001: 512}	1000	1000 (**MATCH**)
Swapped	1000	`[1, 0, 2, 3]`	{0001: 512}	1000	0100 (**MISMATCH**)

```
run_a = service.simulate_adapter("1000", [0, 1, 2, 3]) # -> reported '1000' (
    MATCH)
run_b = service.simulate_adapter("1000", [1, 0, 2, 3]) # -> reported '0100' (
    MISMATCH)
```

Phase E: Permutation Exploit & Reachability Bound (2:30–3:15)

Attacker Objective: Given a source oracle s, craft a valid permutation $\pi \in S_n$ so that $r_j = s_{\pi(j)}$ spoofs a target fingerprint t.

```
def craft_compatibility_map(source: str, target: str):
    if len(source) != len(target) or source.count("1") != target.count("1"):
        return None  # Invariant: permutations preserve Hamming weight!
    zeros = [i for i, b in enumerate(source) if b == "0"]
    ones  = [i for i, b in enumerate(source) if b == "1"]
    return tuple(zeros.pop(0) if b == "0" else ones.pop(0) for b in target)
```

The Hamming-Weight Invariant

Because a permutation only reorders bit positions without flipping bit values, $\sum_j s_{\pi(j)} = \sum_i s_i$. Thus, t is reachable from s **if and only if** $\mathrm{wt}_H(s) = \mathrm{wt}_H(t)$:

- **Reachable ($\mathbf{wt}_H = 2 \to 2$):** `source = '1100'` $\to$ `target = '0101'` via $\pi =$ `(2, 0, 3, 1)` (`exploit_succeeded = True`).
- **Unreachable ($\mathbf{wt}_H = 2 \to 3$):** `source = '1100'` $\to$ `target = '1110'` returns `None` (`is_impossible = True`).

Phase D: Canonical Defense & Evaluation (3:15–4:00)

Design Principle: Legacy presentation helpers may remain for UI display, but authorization **must** compare canonical $(q_0 \ldots q_{n-1})$ bits directly against `registry_fingerprint`.

```
def secure_decode(counts: dict[str, int]) -> str:
    if not counts:
        raise ValueError("Measurement counts dictionary cannot be empty.")
    return canonical_bits_from_counts(counts)  # dominant_key(counts)[::-1]

def secure_decision(counts: dict[str, int], registry_fingerprint: str) -> str
    :
    decoded_fingerprint = secure_decode(counts)
    _validate_bits(registry_fingerprint, n=len(decoded_fingerprint))
    return "MATCH" if decoded_fingerprint == registry_fingerprint else "
        MISMATCH"
```

Defense Test Suite Results: 13/13 Passed (0 Failed)

- 4/4 **Instance Cases:** `CAL-ALPHA`, `CAL-BETA`, `MIG-DELTA`, `INCIDENT-7` → all `MATCH`.
- 5/5 **Unseen Positives:** `0000`, `1111`, `0101`, `1011`, `0010` under non-identity maps → `MATCH`.
- 4/4 **Unseen Spoof Attacks (Negative Tests):** Wrong oracle + crafted spoofing map (e.g., oracle `0101` vs. registry `1100` with `[3,1,2,0]`) → all rejected with `MISMATCH`.

Research Extensions: Orbit-Stabilizer, $n = 16$ & Metamorphic (4:00–4:35)

1. **Exhaustive 16×24 Orbit-Stabilizer Analysis (384 runs):**

Hamming Weight w	$w = 0$	$w = 1$	$w = 2$	$w = 3$	$w = 4$
Preserving Perms $\lvert\text{Stab}(x)\rvert = w!(4-w)!$	24	6	4	6	24
Altering Perms $24 - w!(4-w)!$	0	18	20	18	0
Reachable Targets $\lvert\mathcal{O}(x)\rvert = \binom{4}{w}$	1	4	6	4	1

2. **n-Bit Generalization & Scalability ($n = 16$):**
   - Brute-force permutation search scales as $\mathcal{O}(n!)$ ($16! \approx 2.09 \times 10^{13}$), whereas our $\mathcal{O}(n)$ index-matching exploit solves $n = 16$ instantaneously (`valid_permutation_found = True`).
   - Quantum BV oracle uses 1 query ($\mathcal{O}(n)$ CX depth), while classical statevector simulation scales as $\mathcal{O}(2^n)$.
3. **Blind Metamorphic Testing:**
   - Tests Report(C, π_A) == Report(C, π_B) without knowing the secret fingerprint: catches legacy adapter (2/3 violations) vs. `secure_decode` (0/3 violations).

Claim Sheet, Assumptions & Limitations (4:35–5:00)

Claim	Experimental Evidence	Trusted Assumption	Honest Limitation
1. Not quantum noise	512/512 deterministic shots; `Statevector` $p = 1.0$.	Ideal simulator (`BasicSimulator`).	Real NISQ hardware needs a majority-vote threshold.
2. Circuit encodes registry	Reversing raw keys yields `1010` & `1100` matching registry.	Slot $i \leftrightarrow q_i$; `build_bv_probe` intact.	Cannot detect if the quantum oracle itself is replaced.
3. Metadata alters legacy decision	Swapping `[0,1,2,3]` $\leftrightarrow$ `[1,0,2,3]` on 1000 flips `MATCH/MISMATCH`.	Legacy adapter runs before comparison.	Bounded by Hamming weight $\mathrm{wt}_H(s) = \mathrm{wt}_H(t)$.
4. Mitigation generalizes	Passes 13/13 tests, 384 exhaustive, $n = 16$, metamorphic.	Registry & raw `counts` are authentic.	Does not stop direct memory tampering of `counts`.

Thank You! Ready for Jury Questions & Live Perturbations.